Parallelizing Multiphase CFD

Mihir Kamat
mskamat@ncsu.edu
North Carolina State University
Raleigh, North Carolina, USA

Abstract

This project covers parallelization of a computational fluid mechanics solver using Schwarz domain decomposition with coarse correction. The results obtained by this method and the analysis of those results are also covered. You can find the project here: https://github.com/kamatmihir2002/fluidsim/tree/main

Keywords

Parallelization, Fluids, Schwarz, Coarse Domain, Domain Decomposition

ACM Reference Format:
Mihir Kamat. 2018. Parallelizing Multiphase CFD. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 5 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Multiphase Computational Fluid Dynamics is used in many day-to-day applications. For example, weather simulation can give an idea as to what sort of local climate a place will have at a certain time. Multiphase CFD allows us to examine more real-world influences over traditional CFD examples like the effect of suspended high-altitude water droplets (clouds) on airfoils or on jet engines.

This project report describes in detail the methodology of the original problem we are trying to solve. As CFD is a non-trivial problem to parallelize due to the interdependence of the velocity and the pressure fields on their neighboring cells, the method of parallelization will also be described in detail, including the domain-decomposition methods used. Results obtained by using these methods and analyses on the particular results will also be discussed in detail.

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

2 Methodology

2.1 The Equations

The Navier-Stokes Equations for an incompressible fluid are denoted as follows:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\,\mathbf{u} = -\frac{1}{\rho}\,\nabla p + \nu\,\nabla^2 \mathbf{u} + \mathbf{f}, \tag{1}$$

$$\nabla \cdot \mathbf{u} = 0, \tag{2}$$

The first equation describes the momentum of the fluid at each discrete point in the grid, and the second equation is the incompressibility constraint for the fluid (i.e. no fluid is created or destroyed). To enforce the second constraint, the Pressure Poisson equation is solved:

$$\nabla^2 p = \frac{\rho}{\Delta t}\,\nabla \cdot \mathbf{u}^*, \tag{3}$$

This equation is obtained by taking the divergence of equation (1) and substituting into it equation (2).

2.2 Solving the Navier Stokes Equations computationally

To solve differential equations computationally, they are discretized into a grid, and the equation is solved on each point of the grid. This is called the finite difference method. For this particular problem, the steps for solving these equations for every step would look like this:

Algorithm 1 Navier Stokes, Single Step

```
procedure NavierStokes( dt )
    error ← 1
    while 0 ← error do
        p ← pressurePoisson()
        error ← getPoissonError()
    end while
    u ← MomentumConserve(p, dt)
    return u, p
end procedure
```

By its nature, finite difference methods are intuitively parallel. But in this particular case, difficulty arises due to the neighboring dependencies of the velocity and the pressure fields (due to the pressure and velocity field divergences). To deal with this problem, domain decomposition methods are used.

Unpublished working draft. Not for distribution.

2.3 Domain Decomposition

Algorithm 2 Schwarz Domain Decomposition with Coarse Correction

1: **procedure** SCHWARZDD(Problem P, Domain D)
2: $d[1..L] \leftarrow GetSubDomains(P, D)$
3: $d^{coarse} \leftarrow GetCoarseDomain(P, D)$
4: $error \leftarrow []$
5: $averageError \leftarrow 1$
6: **while** $0 \leftarrow averageError$ **do**
7: **for** domain[i] in d[1..L] **do**
8: $p[i] \leftarrow solveEquation(P, domain[i])$
9: $error[i] \leftarrow getError(P, domain[i])$
10: **end for**
11: $averageError \leftarrow mean(error)$
12: $MatchInterfaces(p[1..L], d[1..L])$
13: $p^{coarse} = Restrict(d^{coarse} \leftarrow d[1..L])$
14: $p_k^{coarse} = p^{coarse}$
15: $p_{k+1}^{coarse} \leftarrow solveEquation(d^{coarse})$
16: $Prolongate(p[1..L] \leftarrow p_{k+1}^{coarse} - p_k^{coarse})$
17: **end while**
18: **return** p
19: **end procedure**

Domain decomposition is a way to solve a differential equation by breaking the domain of the differential equation into different subdomains and solving them such that, put together, the solution for the original domain is obtained. There are various methods for domain decomposition that can be used for this particular problem. Here, a Schwarz domain decomposition[3] with coarse correction has been used.

The algorithm is shown above, at the start of this section. A coarse correction step is used to rectify the low frequency errors, which helps the higher frequency, fine-domain iterations to converge faster. The restriction operator samples values from finer domains to inject into the coarse domain, and the prolongation operator accumulates the values from a single point in the coarse domain to all the values in the corresponding fine domain.

In this case, the restriction operator is simply the mean solution of the subdomain, and the prolongation operator is a simple iteration difference. But there are better choices for restriction and prolongation operators, which would lead to faster convergence, such as linear interpolators.

2.4 Using Domain Decomposition as Parallelization method

For a parallel solve, there is not much that needs to be changed in the above mentioned Algorithm 2 – Steps 7 to 9 describing fine subdomain solves can be blindly parallelized. Additionally, subdomain and coarse domain computation can also be precalculated.

The algorithm itself does not have much changed except parallelization of the core fine domain solve. However, the operators themselves will be handled differently for different parallelization methods. The top-level parallel algorithm is shown next:

Algorithm 3 Parallel Schwarz Domain Decomposition with Coarse Correction

1: **procedure** $SchwarzDD^{Parallel}$(Problem P, Coarse Domain d^{coarse}, Fine Domains $d[1..L]$)
2: $error \leftarrow []$
3: $averageError \leftarrow 1$
4: **while** $0 \leftarrow averageError$ **do**
5: **for Parallel** domain[i] in d[1..L] **do**
6: $p[i] \leftarrow solveEquation(P, domain[i])$
7: $error[i] \leftarrow getError(P, domain[i])$
8: **end for**
9: **Barrier**()
10: $averageError \leftarrow mean(error)$
11: $MatchInterfaces(p[1..L], d[1..L])$
12: $p^{coarse} = Restrict(d^{coarse} \leftarrow d[1..L])$
13: **Barrier**()
14: $p_k^{coarse} = p^{coarse}$
15: $p_{k+1}^{coarse} \leftarrow solveEquation(d^{coarse})$
16: $Prolongate(p[1..L] \leftarrow p_{k+1}^{coarse} - p_k^{coarse})$
17: **Barrier**()
18: **end while**
19: **return** p
20: **end procedure**

2.5 Adapting to Distributed Machines

For a multi-threaded approach on a single machine, again there are no changes in the operators described. To adapt this method to fit into a distributed computing approach, some changes are made:

- The coarse restriction, prolongation and solve would take place in a single rank, whereas the fine domain solves and error calculations would take place on all ranks.
- The interface exchange would also take place on all ranks, but care has to be taken for cases when the subdomain is on a boundary.
- Errors would also be gathered from all ranks into the same rank where the coarse problem is being solved. However, as the errors are computed in parallel along with the fine solve itself, this step does not involve much computation.

For the interface matching, for every fine domain and its neighbor, the mean of the corresponding interface values between the domain and the neighbor is calculated and substituted back into the appropriate positions in both itself and the neighbor.

2.6 A Particle-Liquid Multiphase Approach

To introduce multiphase dynamics into a Navier-Stokes fluid, a set of discrete particles P is introduced such that:

$$P = \{p_i = \langle \vec{x}_i, \vec{v}_i, \vec{a}_i, m_i, C_i \rangle\} \tag{4}$$

where p_i describes the state of a particle with mass m_i, velocity v_i, position x_i, acceleration a_i and drag C_i. To calculate the effect of the fluid on a particle, the acceleration of a particle p_i is calculated using a simpler version of the MP-PIC model[4]:

$$\vec{a}_i = C_i \left(\vec{u}_{fluid} - \vec{v}_i\right) - \frac{\nabla \cdot p_{fluid}}{m_i} \tag{5}$$

Unpublished working draft. Not for distribution.

After this, the particle velocities and positions are integrated normally with each timestep. To calculate the force exerted on the fluid by the particles, a force field on the fluid grid is composed like so:

$$F_{cell} = \sum m_i \vec{a}_i \text{ for all } p_i \in cell \tag{6}$$

As the fluid momentum conservation equations themselves have a force term, substituting the obtained force field in that place solves our multiphase problem.

2.7 The Particle System Algorithm

The steps described above would look like this algorithmically:

Algorithm 4 Particle System, single step

```
procedure ParticleSystem( Particle System P, Fluid F, dt )
    exchangeMomentum(P ← F)
    checkBoundaries(P)
    exchangeForces(F ← P)
    for p_i in P do
        v_i = v_i + a_i dt
        x_i = x_i + v_i dt
    end for
end procedure
```

As with the fluid, the particles are also parallelized. In a similar fashion, our full particle system is broken into subdomains, but there is no coarse domain to handle here. Instead, for every time step, the particles out of bounds of a particular domain are taken into account and transferred to the appropriate domain.

Algorithm 5 Parallel Particle System, single step

```
procedure ParticleSystem^parallel( Particle Systems P[1..L],
Fluid Domains F[1..L], dt )
    for Parallel P[i] in P[1..L] and F[i] in F[1..L] do
        exchangeMomentum(P[i] ← F[i])
    end for
    checkBoundaries(P)
    for Parallel P[i] in P[1..L] and F[i] in F[1..L] do
        exchangeForces(F[i] ← P[i])
        for p_i in P[i] do
            v_i = v_i + a_i dt
            x_i = x_i + v_i dt
        end for
    end for
end procedure
```

The *checkBoundaries* function here checks if every particle in each particle system $P[i]$ within P is still in that same system. If not, it removes the particle from the current system and inserts it into the appropriate neighboring system.

2.8 Putting it together

Until this point, all the individual sections have been described in detail, along with their algorithms. For this final section in the methodology, the algorithm as a whole is described:

Algorithm 6 Parallel Multiphase CFD solver

```
procedure Main( Particle System P, Fluid F, Domain D)
    d[1..L] = GetSubDomains(D)
    d^coarse = GetCoarseDomain(D)
    p[1..L] = SplitParticleSystem(P)
    t ← 0
    while t < end do
        ParticleSystem^parallel(p[1..L], d[1..L], dt)
        SchwarzDD^parallel(pressurePoisson, d[1..L], d^coarse)
        SchwarzDD^parallel(momentumConserve, d[1..L], d^coarse)
        t ← t + 1
    end while
end procedure
```

3 Results

3.1 Implementation Details

This section details the finer implementation details before the results are discussed and analyzed proper in the proceeding sections.

Parallelization was performed using OpenMP and the source code was compiled using the GCC compiler using the flags `-O3`, `-march=native`, and `-ffast-math`.

3.2 Correctness

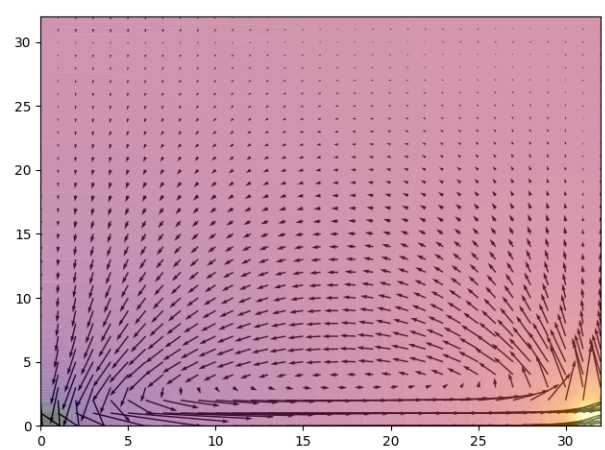
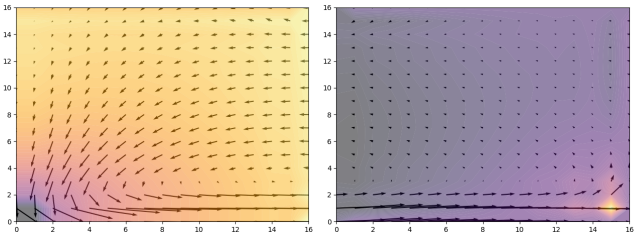


Figure 1: A classic lid-driven cavity

To check for the correctness of the solver, a classic CFD problem is run – the lid driven cavity. This problem describes a container with 3 closed walls and one moving lid at a certain velocity. Characteristics of this problem include a vortex-like appearance in the fluid flow.

Figure 1 denotes a lid-driven cavity visualization on a single thread. For visualization of the lid driven cavity in mutiple domains:

Unpublished working draft. Not for distribution.



(a) (b)

Figure 2: A lid-driven cavity with parallel domain decomposition. (a) and (b) are neighboring domains

In the visualization of the lid driven cavity with domain decomposition, the vortex-like structure is formed at the interface of the two domains as seen in Figure 2. This means our solver is currently solving the problem correctly.

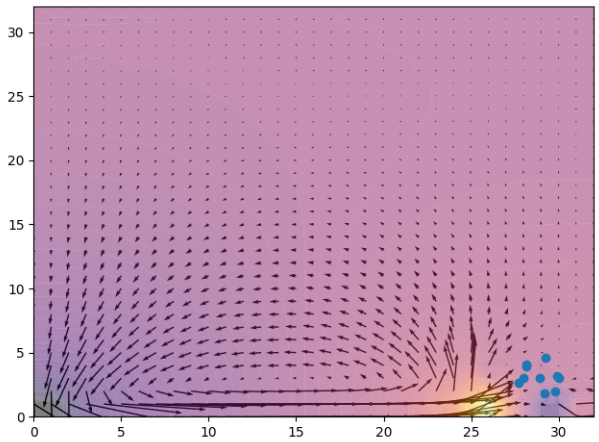


Figure 3: Introducing particles in the lid-driven cavity. Fluid velocity curving around the particle's area of influence is observed.

3.3 Performance

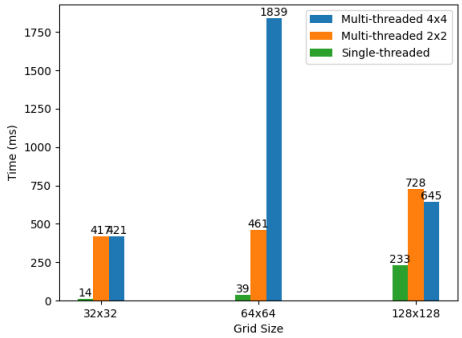
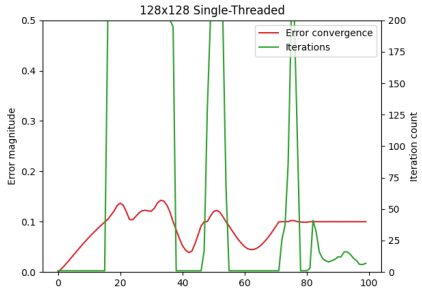


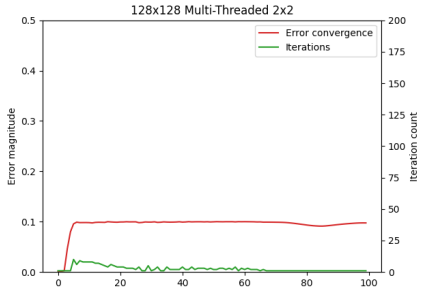
Figure 4: Performance plot of single threaded as compared with Multithreaded 4x4 and 2x2

It is not guaranteed that the parallel implementation will be faster than the serial implementation. Given less than enough threads, the parallel version will almost always be slower than the serial version. In a distributed system, this effect will be more apparent, due to communication costs. Why this is caused is duscussed in detail in the next section.

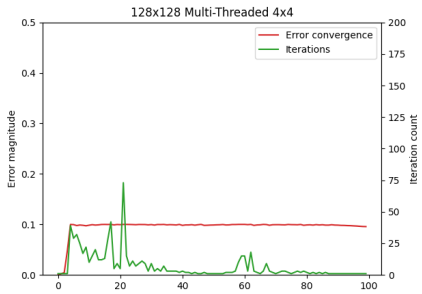
3.4 Analysis



(a)



(b)



(c)

Figure 5: Stability plots for error iterations and magnitudes for the serial solver (a) and the parallel solvers (b, c). All of these plots are for grid extents 0.0 to 2.0.

As stated in the previous section, there is quite a steep drop in performance for the parallel solvers as compared to the serial solver. This is due to a function in the parallel solver that does not exist in the serial solver – the $exchangeInterfaces$ function. Hence it

Unpublished working draft. Not for distribution.

becomes quite obvious that the interface exchange is the bottleneck here, and to curb this bottleneck, better domain division strategies are needed – dividing domains such that interface size is minimized and the internal subgrid area is maximized. For this particular parallel version, the area is naively divided into square sub-grids, hence it is not effective in speeding up the algorithm.

However, when compared to the serial algorithm in terms of error stability and convergence, the parallel algorithm showed promise. The current parallel and serial solvers are iterative – they iteratively re-solve a problem repeatedly while converging closer to the true solution. However, if the $Courant - Friedrichs - Lewy$ condition is not met, the error fails to converge and the solvers are unstable and fail to correctly solve for the next timesteps.

The $Courant - Friedrichs - Lewy$ condition states that, for a fluid simulation at a certain domain resolution, the timestep has to be below a certain threshold. In our solver, the timestep is fixed, but the resolution can be changed. The parallel subgrid resolution, in turn, is smaller than the original grid resolution by a factor of parallelization degree. This leads to better stability at quite high grid sizes, hence increasing the scalability of our parallel solver and allowing it to handle larger loads.

4 Conclusion

This paper has investigated a parallel approach to multiphase computational fluid dynamics, and investigated and analysed the results. In conclusion, to get a timing speed-up, more advanced domain splitting strategies need to be applied, in order to reduce the bottlenecks formed during the interface exchange. However, the solver itself is more scalable when parallelized, due to better conformance with the $Courant - Friedrichs - Lewy$ condition in every subdomain, which leads to better overall error stability.

5 The Future of This Project

As stated, future steps that can be taken to improve this solver are to employ better domain splitting strategies. Another way this can be improved is to employ not just better domain splitting strategies, but better domain decomposition strategies as a whole. Quite a lot of advanced domain decomposition strategies have been devised that are better than the one used here such as BDDC (Balancing Domain Decomposition with Constraints) and FETI (Finite Element Tearing and Interconnect) that give better convergence results than the present method. Algebraic Multigrid methods can also be used, where the problem is recursively solved from a coarse grid resolution down to finest grid resolution.

6 References

[1] "A guide to writing your first CFD solver", Mark Owkes, Montana State University, https://www.montana.edu/mowkes/research/source-codes/GuideToCFD.pdf

[2] "12 Steps to Navier Stokes", Lorena Barba, https://lorenabarba.com/blog/cfd-python-12-steps-to-navier-stokes/

[3] "Domain Decomposition Methods – Algorithms and Theory", Toselli & Widlund, 2005

[4] "The multiphase particle-in-cell (MP-PIC) method for dense particulate flows", M.J. Andrews, P.J. O'Rourke

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

Unpublished working draft. Not for distribution.